Final Report on Analyzing Survey Data on Mortgage Originations

SIADS 696: MADS Milestone II

Michael Schoose
Paul Stotts
John Papazian

June 25, 2025

https://github.com/Papazian/MADS_Milestone_II

Introduction

The goal of our project is to better understand the relationship between a borrower's characteristics and their riskiness, which is expressed as a function of their mortgage interest rate spread at origination. We use the National Survey of Mortgage Originations (NSMO).[1] This dataset is based on a five percent sample of new residential mortgages. The Federal Housing Finance Agency (FHFA) completes monthly surveys of the mortgage market, and they collect data on the characteristics of individual mortgages. We use this dataset for both supervised learning and unsupervised learning.

For our investigation, we create a synthetic target variable of "Beta" that represents the riskiness of a borrower. Our synthetic target variable of Beta is analogous to the Beta (β) used in the Capital Asset Pricing Model (CAPM). This financial formula defines the expected return on an equity investment (Re) as a function of the risk-free rate (Rf) plus the Beta of the investment multiplied by the equity risk premium. The equity risk premium itself is defined as the expected rate of return of the market (Rm) minus the risk-free rate (Rf). The expected rate of return of the market (Rm) is akin to the return of a diversified portfolio of stocks that matches the S&P 500 index. Likewise, one could think of the risk-free rate (Rf) as the yield of highly rated government bonds such as the 30-year U.S. Treasury, which has a very low probability of default.

$$Re = Rf + \beta(Rm - Rf)$$

$$\beta = (Re - Rf) / (Rm - Rf)$$

In the Capital Asset Pricing Model, Beta (β) is a measure of the volatility or risk of an investment in relation to the market overall. Investments with a Beta of 1 have an expected risk equivalent to the risk of the market overall. Investments with a Beta greater than 1 are more risky than the market overall, while investments with a Beta less than 1 are less risky than the market overall. There is an underlying connection between the risk of an investment and the investment's expected rate of return. Investors are willing to tolerate a high-risk investment if the investment is expected to have a rate of return higher than the market overall. Likewise, investors will tolerate an investment with a low rate of return if it is expected to have a lower risk than the market overall. Thus, Beta (β) is useful to distinguish a high-risk investment with a high expected rate of return versus a low-risk investment with a low expected rate of return.

In a similar manner, the CAPM formula can be applied to create a Beta for a specific mortgage. Banks view mortgages similar to how investors view investments. They lend money to homeowners with the expectation of getting a return on their investment. Some homeowners have a higher risk of defaulting on their mortgage. Thus, banks give those homeowners a higher interest rate on their mortgage to compensate the bank for taking on a higher risk. We use the following variables in our data to create the synthetic target variable of Beta that represents the riskiness of a borrower.

treasury_yield = market yield on U.S. Treasury securities at 30-year constant maturity[2]
pmms: Freddie Macs' primary mortgage market survey rate at origination[3]
rate_spread: mortgage interest rate spread at origination

We can conceptualize the treasury_yield as the risk-free rate (Rf). Likewise, we can conceptualize the pmms as the expected risk of the market overall (Rm) because it is an average rate of all mortgages surveyed. Our expected rate of return for any individual mortgage can thus be defined as:

$$Re = rate\_spread + pmms$$

In other words, our expected rate of return for any individual mortgage is the sum of the mortgage interest rate spread at origination plus Freddie Mac's average mortgage rate from their market survey. We can substitute these terms into the CAPM formula to compute the Beta for each individual mortgage:

$$\beta = (Re - Rf) / (Rm - Rf)$$
$$\beta = (rate\_spread + pmms - Rf) / (Rm - Rf)$$
$$\beta = (rate\_spread + pmms - treasury\_yield) / (pmms - treasury\_yield)$$

Using this formula, we compute Beta (β) as the synthetic target variable for our models. Similar to the Beta for a stock, it represents the riskiness of an individual mortgage compared to other mortgages. Mortgages with a Beta of 1 have an expected risk equivalent to the risk of the mortgage market overall. Mortgages with a Beta greater than 1 are riskier than the market overall, while mortgages with a Beta less than 1 are less risky than the market overall.

The primary problem that we are trying to solve is to see if we can predict the risk of the mortgages (i.e. the Beta) in the National Survey of Mortgage Originations portfolio using mortgage characteristics as features. We are motivated to work on this research to learn how mortgage characteristics influence systematic risk relative to market movements. Moreover, we are interested to see if we can identify high-beta versus low-beta mortgage profiles for portfolio management. Solving this problem will have an impact because it will enable us to learn what are the underlying variables explaining the riskiness of mortgages. To carry out our research, we are employing both supervised learning and unsupervised learning.

For supervised learning, we are interested to learn which mortgage features (e.g., loan characteristics, borrower profiles, property details) are most predictive of a mortgage Beta. We want to learn how accurately different Machine Learning models can predict mortgage Beta. Moreover, we are curious which model family (probabilistic, tree-based, instance-based, etc.) performs best for this financial prediction task. For unsupervised learning, we want to learn if there are any natural clusters of mortgages based on their characteristics. Moreover, we want to know if these clusters correspond to different risk profiles. Lastly, we are curious to know if we can identify mortgage segments that behave similarly in terms of systematic risk.

Related work

Other scholars have developed Machine Learning models to understand the risk level in mortgages. Sadhwani, Giesecke, and Sirignano (2021) examined the behavior of mortgage borrowers in the United States between 1995 and 2014. They built a deep learning model of multi-period mortgage delinquency and foreclosures. However, they did not use traditional statistical methods such as Ordinary Least Square (OLS) linear regression. Rather, they focused on building just neural network models.[4] We chose to use traditional statistical methods in conjunction with Machine Learning methods. Also, we are not predicting mortgage delinquency nor foreclosure, but rather the riskiness of an individual mortgage as expressed by its Beta.

Pillai, Woodbury, Dikshit, Leider, and Tappert (2019) developed Machine Learning models to predict the approval or denial of mortgage applicants. They used a two-tier machine learning model based upon risk exposure from the macro and micro level. They explore a variety of classification models including random forest, gradient boosting, and logistic regression.[5] Our target variable of Beta is continuous. Therefore, we explore regression models rather than classification models.

Lastly, Ojha, and Lee (2021) examined the U.S. Residential Mortgage Backed Securities (RMBS) to predict the probability of default. They studied the Fannie Mae loan data that includes around 1.5 million mortgage loans originating from 2005 to 2009 in the United States. They investigated the probability of default in terms of credit

risk based upon loan origination and performance characteristics. Their research focused on different types of neural networks. They developed models using Convolution Neural Network (CNN), Recurrent Neural Network (RNN), and Long Short-Term Memory (LSTM).[6] In contrast, we would like to focus on traditional Machine Learning approaches such as gradient boosting and random forest.

Data Sources

We use the National Survey of Mortgage Originations (NSMO), which is based upon the time period of 2013 to 2023.[7] This survey data is a subset of the National Mortgage Database (NMDB), which is managed by the Federal Housing Finance Agency (FHFA) and the Consumer Financial Protection Bureau (CFPB).[8] These government agencies have provided information about the American mortgage market, which is based on a five percent sample of all residential mortgages. The FHFA completes a monthly survey of the mortgage markets, and they collect data on the characteristics of individual mortgages. These include subprime and nontraditional mortgages. In addition, FHFA collects information on the creditworthiness of borrowers. The National Mortgage Database includes information on Vantage score, which is a proxy of FICO credit scores.

The NMDB is a de-identified loan-level database of closed-end first-lien residential mortgages. It represents the market as a whole and contains detailed loan-level information on the terms and performance of mortgages. It also includes characteristics about the associated borrowers and properties. It has a historical component that dates back before the financial crisis of 2008, and it has been updated over time.

One challenge of this project is that the National Survey of Mortgage Originations is rich with over 50,000 observations and over 500 features.[9] Many of the survey questions are categorical in nature. We first conduct exploratory data analysis. Afterwards, we subset the data and reduce the number of features before conducting supervised learning and unsupervised learning.

Feature engineering

We did a lot of initial preprocessing before doing feature selection. After reading in the raw CSV survey data, we first bifurcate the variables into categorical variables and numeric variables. This is necessary because they have different approaches to imputation. We clean the raw data by converting negative values (representing missing values) into null values. Next, we join our mortgage survey data with U.S. Treasury yields downloaded from the Federal Reserve. We compute the average 30-year U.S. Treasury yields over each year and month in our survey data, which we use as the risk-free rate (Rf). Using the modified CAPM formula above, we calculate the Beta for every mortgage. As discussed above, this Beta value serves as our synthetic target variable. There are many outlier values for Beta. Therefore, we Winsorize these Beta values at the 5th and the 95th percentile to reduce the influence of outliers. Lastly, we do one-hot encoding by creating dummy variables representing each category for each categorical variable.

We also engineer a new feature. There are many Vantage scores in the mortgage survey, which is a proxy of FICO credit scores. Banks typically use a “lower middle” credit score. Banks usually pull from the three main credit bureaus for each borrower and then take the middle score for each borrower on the mortgage (e.g., the spouse). We use a similar approach to engineer a ‘low_score’ feature, which represents the lowest of the borrower’s middle score.

Using subject matter expertise, we decided to exclude certain variables. These variables have been excluded for reasons such as being non-relevant identifiers, components of the target variable, or data from after the mortgage

origination providing “data leakage”. The excluded variables include the Vantage score after mortgage origination, mortgage performance status over time, forbearance status over time, etc. It is important to remove these variables because they are not useful to predict the Beta for each mortgage at origination.

Finally, we do a train_test_split() to partition 80% of our observations for model training and keep the remaining 20% as holdout for model validation. We impute missing values for the numeric variables using the mean value imputation. The categorical variables do not need to be imputed because they already have categories representing missing values. We fit a SimpleImputer() on just the training data, which we then apply to both the training partition and the validation partition. We also fit a StandardScaler() on just the training data, which we then apply to both the training partition and the validation partition. These steps are necessary to prevent “data leakage” of information.

Part A. Supervised Learning

Methods Description:
The Supervised Learning workflow began with importing the sanitized and split training and test data into defined variables. The dataset was quite large (greater than 50k rows), and so we added a toggle to subset the data. This was helpful in quick iteration through model exploration and training while still giving us the option of processing the entire dataset periodically to validate the results. The toggle also allowed us to choose what percentage of the data to process, which allowed us to test performance at various sample sizes. The last step before model training and selection was to import the variable label files that were output of the data preprocessing file.

Feature Representation:
All categorical features were one-hot encoded as part of data cleaning, expanding the feature set with dummy indicator columns. This means our feature matrix includes a mix of continuous numeric fields and numerous binary dummy variables representing categories (e.g. loan types, survey responses, etc.). No additional feature scaling was applied to the numeric features globally since tree-based models and linear models can handle raw values. However, we did standardize features when required by a specific algorithm (notably k-NN) to ensure meaningful distance calculations. Overall, the feature set captures various aspects of the loan applications: credit scores, loan-to-value (LTV) ratios, debt-to-income (DTI) ratios, loan structure details (term, loan type flags), borrower behavioral indicators, neighborhood indices, and future intent variables.

Model Selection and Rationale:
We explored three different regression algorithms to fulfill the project requirements and provide a balance of interpretability and complexity.

- **Lasso Regression (Probabilistic Linear Model):** We included a Lasso (L1-regularized linear regression) to serve as a baseline and to leverage its ability to perform feature selection. Lasso can shrink less-informative feature coefficients to zero, which is useful given the high-dimensional feature space after one-hot encoding. This model is fast to train and its coefficients offer insights into which features have the strongest linear relationship with the target. However, as a linear method, it may underfit complex nonlinear patterns.

- **k-Nearest Neighbors Regression (kNN - Instance-Based Model):** We chose k-NN as a simple non-parametric method that makes predictions based on the outcomes of similar instances in the training data. The k-NN model can capture local nonlinear relationships that a global linear model might miss. It serves as a straightforward benchmark for nonlinear modeling. The downside is that k-NN can struggle with high-dimensional data (distance becomes less meaningful), and it can be computationally expensive for large datasets. We mitigated some issues by scaling features within a pipeline so that no single feature

dominates the distance metric. Still, we expected k-NN to be less accurate than more sophisticated models on this task.

- **Gradient Boosting Ensemble (HistGradientBoostingRegressor - Tree Based Model):** For a more powerful predictive model, we employed an ensemble tree-based method. We used the histogram-based gradient boosting regressor from scikit-learn, which is well-suited for large datasets and can automatically handle missing values and categorical splits efficiently. Gradient boosting iteratively builds an ensemble of decision trees, optimizing to reduce error, and can capture complex interactions among features. We anticipated this model to yield the best accuracy given enough tuning, at the cost of reduced interpretability and longer training time. This choice aligns with the expectation that ensemble methods often perform well on structured data and was included to meet the requirement of exploring an advanced model.

Hyperparameter Tuning Strategy:
Each model's major hyperparameters were tuned via a two-stage approach, using cross-validation for reliable performance estimation. In the first stage, we performed a broad search for good hyperparameters for each model type using randomized search with 5-fold cross-validation. This fast RandomizedSearchCV allowed us to efficiently sample the hyperparameter space without evaluating every combination, which is especially useful for the more complex models. All searches used the same 5-fold shuffled K-Fold CV strategy and optimization criterion (minimizing root mean squared error). We embedded the k-NN model in a pipeline with a standard scaler during tuning to ensure the distance calculations were based on standardized feature values.

After the randomized search, we examined the cross-validation results to identify the best-performing model type. The initial results indicated that the Gradient Boosting model had the lowest cross-validated error (outperforming Lasso and k-NN), so we selected it for further refinement. In the second stage, we carried out an exhaustive GridSearchCV on the Gradient Boosting model, focusing on a more refined grid around the promising hyperparameter values found in Stage 1. This grid search tried all combinations in a specified grid (learning rate values around 0.02 - 0.04, max_depth around 10–20, max_iter around 500–1500) to fine-tune the model. The grid search also used 5-fold CV and aimed to squeeze out the optimal settings for the booster. The best hyperparameters discovered for the gradient boosting regressor were: a learning rate of 0.03, maximum tree depth of 15, 500 boosting iterations (trees), a max of 63 leaves per tree, a minimum of 10 samples per leaf, and a small L2 regularization of 0.0005. These settings offered a good balance of bias and variance for our dataset. We then refit the Gradient Boosting model on the entire training set with these tuned parameters to serve as our final model. Throughout this process, we saved each model from the random search stage as well, both for comparison and to use in plotting learning curves.

Supervised Evaluation:
We evaluated model predictions using several complementary metrics appropriate for regression. The RMSE was our primary metric for model selection, as it penalizes larger errors and aligns with the goal of minimizing squared error losses. We also report MAE (mean absolute error) to gauge average absolute deviation (less sensitive to outliers than RMSE) and R-squared (coefficient of determination) to measure the proportion of variance in Beta explained by the model. These metrics provide a holistic view of performance – error magnitudes in the original Beta units and relative explanatory power. Given that Beta prediction is a difficult task, we expected relatively high errors and low R-squared, so these metrics help confirm how much better our models are than a naïve baseline.

Table 1 summarizes the models' predictive performance based on 5-fold cross-validation and the hold-out test results. The grid-tuned gradient boosting model achieved the best CV score with a mean RMSE 0.5871, slightly outperforming the random-tuned gradient boosting model and substantially outperforming Lasso and k-NN with RMSE scores of .6288 and .6627 respectively. This indicates the tree-based ensemble captured patterns that the linear and instance-based models missed. The variance across folds was moderate for the gradient boosting models,

suggesting its performance was consistently better. Lasso and k-NN showed higher average CV error; in fact, their CV RMSE were nearly equal, reflecting that both a heavily regularized linear model and a local neighbor-based model had similar predictive power on this task. Overall, cross-validation pointed to gradient boosting as the top performer by a wide margin in terms of error minimization.

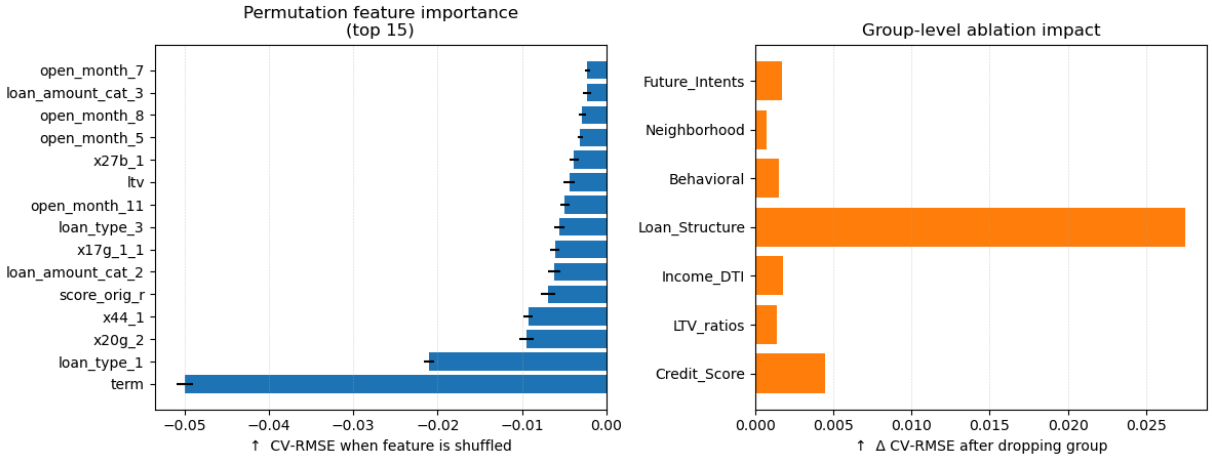
Model	MAE	RMSE	CV_RMSE_mean	CV_RMSE_sd
GB (tuned – grid)	0.3994	0.5871	0.5826	0.0028
GB (tuned – random)	0.4	0.5881	0.5831	0.003
Lasso (tuned – random)	0.4323	0.6288	0.6219	0.0031
k-NN (tuned - random)	0.4642	0.6627	0.6578	0.0047

Table 1 - Cross Validation Performance

Feature Importance and Ablation Analysis:
To interpret the final Gradient Boosting model, we examined feature importance using permutation importance and conducted a group-wise ablation study. We computed permutation importances on the validation data for all features and found that the most influential features included various loan structure attributes and credit-related variables. For example, the loan term (such as 30-year vs 15-year) and the loan amount emerged among the top drivers of the model's predictions. Credit indicators like credit score and LTV ratios were also high on the importance list. Even the third and fourth highest features of importance were x20g_2 (*In the process of getting this mortgage from your mortgage lender/broker, has your Loan Estimate revised to reflect changes in your loan terms*) and x44_1 (*Does this mortgage have an adjustable rate*) were reflective of the actual technical details of the loan itself.

We also performed a group ablation analysis to understand the importance of entire feature groups. We defined thematic groups of features (such as: all credit score related features, all LTV ratio features, all income/DTI features, loan structure features, etc.) and then measured the performance drop when each group was removed from the model. This analysis revealed that removing the Loan Structure group (which includes features like loan term, rate type, loan type indicators, and whether the loan is jumbo) caused the most significant increase in error. In fact, the model's cross-validated RMSE rose by roughly 0.02 when loan-structure-related features were dropped, the largest jump among all groups. This highlighted that these loan terms and conditions are critical in predicting the target Beta. The next most important group was the Credit Score group: excluding credit score and related variables led to a noticeable performance degradation. This confirms that a borrower's creditworthiness is also a key predictor. The LTV_ratios group had a smaller but non-trivial impact (removing LTV features slightly hurt performance, with a slight error uptick of a few thousandths). On the other hand, dropping groups like Income/DTI or the borrower's Behavioral indicators or Future Intents had little to no effect on RMSE and the lines barely moved in those ablations. This would suggest that the model found debt-to-income and income data, as well as those survey-based intent fields to not be predictive for the target. The Neighborhood group (geographic and market indicators) also showed minimal impact when removed. Overall, the feature importance and ablation studies consistently indicate that the structure of the loan (how it's structured and priced) and the borrower's credit standing are the dominant drivers of the outcome, but borrower-provided behavioral responses or future plans did not significantly influence the model's predictions. These insights help us validate that the model is using intuitively important factors (like credit and loan terms) to make decisions, and they also highlight areas (like neighborhood variables) that might be safely pruned.

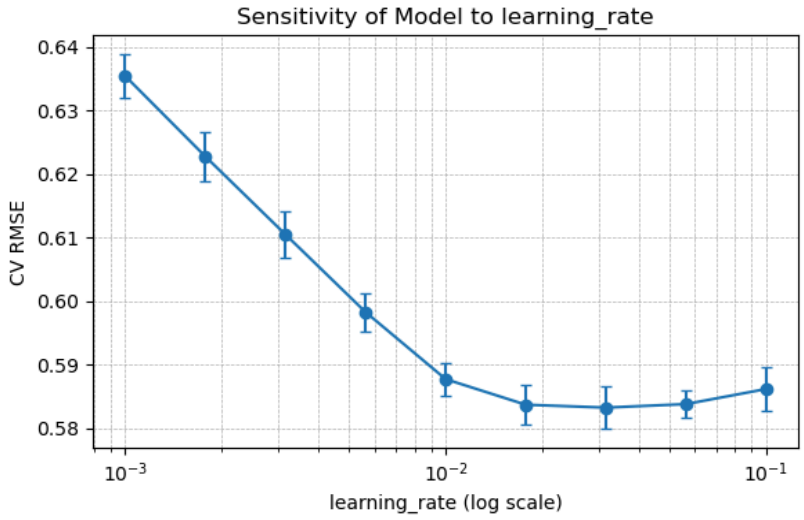
Figure 2: Feature importance and ablation



Hyperparameter Sensitivity:

We also analyzed how sensitive the final Gradient Boosting model is to one of its most critical hyperparameters, the learning rate. Using the tuned model as a baseline, we varied the learning rate across a range (from very low 0.001 up to 0.1, on a logarithmic scale) and observed the effect on cross-validation RMSE. The sensitivity curve (see figure below) shows that extremely small learning rates (0.001–0.005) lead to noticeably higher error, indicating underfitting. This means that the model learns too slowly and cannot converge to a good solution within the fixed 500 iterations. As the learning rate increases into the 0.02–0.05 range, the CV error drops significantly. The performance is best around somewhere between the 0.02 and 0.03 marks, and it remains quite stable for learning rates slightly above or below this value. This suggests the model is not overly fragile to the exact learning rate as long as it is in a reasonable range which is a good sign for robustness.

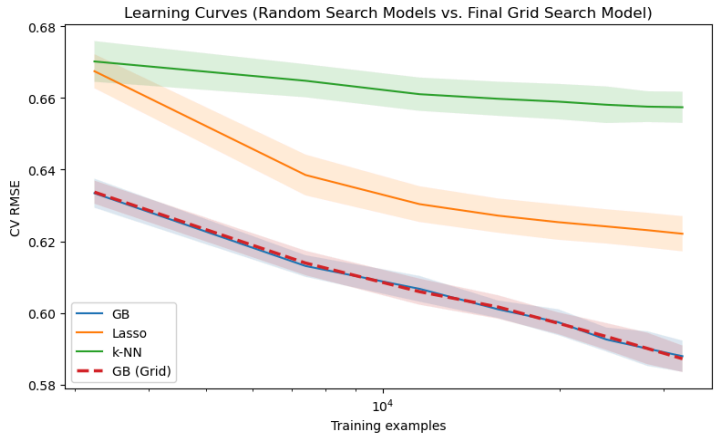
When we push the learning rate to 0.1, we notice a slight increase in error again, which could indicate the model starts to overfit or at least overshoot optimal updates at that higher rate. Overall, the model's accuracy was relatively insensitive to small changes in hyperparameters around the optimum, but extreme changes would noticeably impact performance. This hyperparameter stability implies our tuning choices are in a reasonably flat optimal region, which is desirable for generalization.



Learning Curves:
To further evaluate the models, we plotted learning curves (training set size vs. performance) for each approach using cross-validation. The learning curves (shown in the figure below) compare how quickly each model type gains predictive accuracy as the training set grows, and whether the models might benefit from even more data. For both Gradient Boosting models, the validation RMSE steadily decreases as more training data is used, and it has not plateaued even at the full 40k observations. This suggests that perhaps additional training data could further improve the model, although with diminishing returns.

In contrast, the Lasso model's learning curve leveled off much earlier: by a few thousand samples, its validation error was already near its asymptote. This indicates that the linear model is likely high-bias and cannot take full advantage of more data to significantly improve, since its simplistic functional form limits performance. The k-NN model showed a somewhat intermediate pattern as its performance improved with more data, but it started off with high error on small samples and while it did get better as data increased, the curve was beginning to flatten by the upper end of the data as well. It is worth noting that k-NN had higher variance with smaller training sizes, which reflects its instance-based nature.

When passing the full dataset, the Gradient Boosted model has a clear advantage and the gap between it and the other models continues to widen. These learning curve trends highlight a key trade-off: the more complex model (GB) has a higher capacity and thus continues to improve with additional data, whereas the simpler models (especially Lasso) quickly hit their limit. It also suggests that collecting more data (if possible) would benefit the complex model the most, where simply adding data would not substantially help the linear model.



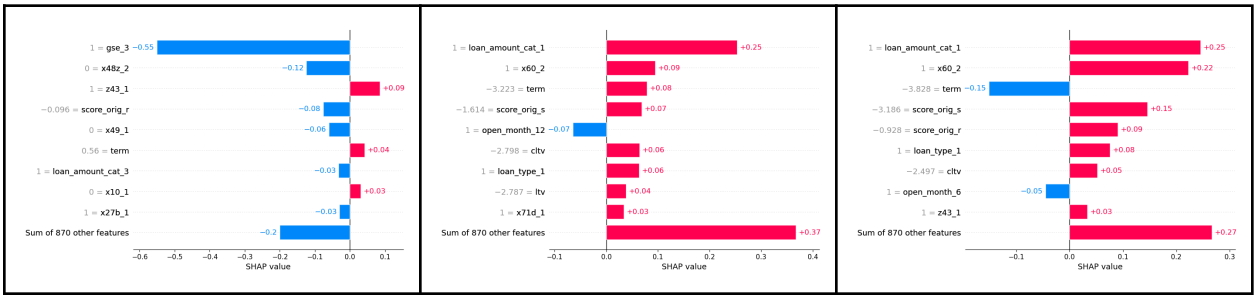
Key Trade-offs
Accuracy vs. Interpretability:
The Gradient Boosting ensemble achieved the highest accuracy but is the least interpretable. Its predictions come from an ensemble of hundreds of trees, making it essentially a black box. On the other hand, the Lasso model is highly interpretable. We can inspect its coefficients to understand the influence of each feature. However, this interpretability comes with notably lower accuracy. K-NN offers instance-based interpretability, but it does not provide a clear global model explanation and is also less accurate than boosting. In this case, we prioritized accuracy for the final model since there was a significant performance gap.

Complexity and Training Time:
This was initially one of our biggest roadblocks until Saurabh Budholiya (the instructional team member assigned to our group) suggested working with a smaller dataset during iteration. This inspired the idea of adding the subset toggle to the beginning of the code workbook. When it came to training the models, the Lasso model is very fast to

train (even with cross-validation) and easy to tune. K-NN is also straightforward with minimal training costs, though finding neighbors at prediction time can be slow. The Gradient Boosting model, however, is very computationally expensive. We mitigated this by using the histogram-based implementation (which is faster than standard gradient boosting) and by using randomized search to narrow down the range and evaluate the base models.

Overfitting vs. Generalization:
We consciously managed model complexity to avoid overfitting. The Lasso inherently guards against overfitting via regularization (if anything, it was more at risk of underfitting). The k-NN model could overfit if 'k' is too low, but our tuning favored a moderate 'k' that provided a balance. The Gradient Boosting model has many knobs that control overfitting. Our final hyperparameters reflect a balance, as demonstrated by the close CV and test errors. We also used cross-validation and early model comparisons to ensure each model generalized well. The results show all models had similar CV and test performance, indicating no severe overfitting in the final choices.

Failure Analysis:
The final supervised-learning checkpoint drills down on the three test-set loans with the largest absolute prediction errors. For each record we compute a full SHAP decomposition, allowing us to see exactly which feature pushes the tuned Gradient Boosting ensemble off target.



Failure #1 – Model bias on an in-range record
The prediction comes in too low even though every variable sits squarely inside its usual range. The heavy downward pull is driven by the gse_3 flag (1, −0.55), supported by x48z_2 (0, −0.12), score_orig_r (−0.096, −0.08) and x49_1 (0, −0.06). Positive lifts from z43_1 (1, +0.09) and term (0.56, +0.04), plus minor nudges from loan_amount_cat_3 (1, −0.03), x10_1 (0, +0.03) and x27b_1 (1, −0.03), cannot offset the aggregate negative contribution (the residual 870-feature sum adds −0.20). Because nothing here is extreme, the miss points to residual bias in this GSE-coded segment rather than data sparsity.

Failure #2 - Outlier / extrapolation cases
An unusual categorical pairing of loan_amount_cat_1 (1, +0.25) and x60_2 (1, +0.09) is amplified by extreme numeric inputs—term = −3.223 (+0.08), score_orig_s = −1.614 (+0.07), cltv = −2.798 (+0.06), ltv = −2.787 (+0.04)—while open_month_12 (1, −0.07) offers the only meaningful brake. The "Sum of 870 other features" adds another +0.37, driving the estimate far above the truth.

Failure #3 - Outlier / extrapolation cases
A near-mirror pattern emerges: loan_amount_cat_1 (1, +0.25) and x60_2 (1, +0.22) headline the uplift. Extreme scores (score_orig_s = −3.186, +0.15 and score_orig_r = −0.928, +0.09) pile on, while term = −3.828 (−0.15) and open_month_6 (1, −0.05) push the other way. Extra lift from loan_type_1 (1, +0.08), cltv = −2.497 (+0.05), z43_1 (1, +0.03) and the residual +0.27 shove the prediction well beyond the observed value.

The in-range miss suggests that deeper trees or data augmentation around similar feature combinations may capture the subtle interactions now being ignored. The two outlier misses show the ensemble's natural extrapolation limits.

Part B. Unsupervised Learning

Methods:
Unsupervised learning was used to analyze the survey questions and answers captured for each sample within the dataset. Clustering was performed primarily through the use of K-means and agglomerative clustering. Principal component analysis (PCA) and t-distributed stochastic neighbor embedding (t-SNE) were utilized for feature reduction before clustering and visual inspection of the clusters. Random forest regression was utilized to identify impactful features within the final clusters and explore the true meaning of the clusters.

Feature Reduction:
The features utilized for this task were the one hot encoded answers to the survey questions for each sample within the dataset. This dataset gave two primary challenges. First, as one hot encoded answers to questions, many of which were binary, linear independence of features was not a safe assumption. Second, the dataset was feature rich and sparse containing 781 different features.

To address the first challenge, questions with binary response options were identified. Then a second filtered dataset was created that dropped one of the two one hot encoded columns associated with each binary variable. This step is critical for the utilization of techniques that assume linear independence of input features such as PCA. Additionally, having multiple one hot encoded features representing both the True and False states can interfere with distance based clustering methods such as PCA. As having both values can be beneficial to tree based methods, the original dataset was retained and used for agglomerative clustering. A few survey questions were also dropped from the dataset after the initial clustering attempts. They were questions that were coded differently over various survey waves, which were leading the clustering algorithms to just identify survey waves. Removing these values reduced the feature space to 517 features.

To address the second challenge, a multitude of approaches were taken to reduce dimensionality prior to performing clustering. Approaches used include PCA and variance thresholds. Clustering was performed on the reduced datasets and clusters were evaluated for performance.

PCA was used to transform the dataset into a reduced feature space before clustering. Scikit-Learn's PCA was utilized. The first PCA approach utilized was transforming the dataset into components that explained 80% of the variance within the original dataset. As this still yielded 249 features, a second PCA decomposition was performed to specifically identify 10 components. Those 10 were more limited and only captured 17.2% of the variance within the dataset. Data normalization was also performed prior to performing PCA as a small number of survey questions were continuous numerical features. Normalization was performed utilizing the scikit-learn StandardScaler.

Variance thresholds were another strategy utilized to reduce the feature space. Scikit-Learn's VarianceThreshold was utilized. Variance thresholds were applied to the 517 feature dataset after removing binary columns. A threshold of 1 was utilized and yielded 228 features. PCA was then applied with a threshold of 80% of variance explained, yielding a dataset with 129 features.

K-Means:
K-Means clustering was performed on each of the reduced feature spaces to identify clusters ranging from n=2 to n=10 clusters using scikit-learn K-Means. Initially, a training loop was utilized that iterated through a range of possible cluster values and compared cluster quality using scikit-learn's calinski_harabasz_score. This failed to yield useful clusters as the training loops either never improved from two clusters, or yielded far too large of cluster numbers before scores became reasonable.

Afterwards, a training loop was performed for each reduced feature set that iterated through a range of clusters from 2 to 10. For each iteration, average silhouette scores were calculated and individual silhouette scores were calculated and plotted for each data point and color was encoded using the cluster labels. Additionally, t-SNE was utilized to reduce the feature space to 2 dimensions. A second visualization was produced for each iteration of the 2-dimensional t-SNE representation and mark color was encoded using cluster labels.

Agglomerative Clustering:
Agglomerative clustering was performed using each of the reduced feature spaces using SciPy Linkage. Cluster distance was calculated using Euclidean distance. Dendrograms were plotted for each dataset's linkage tree to identify the ideal cluster distance to utilize for clustering. Resultant clusters were reviewed for the number of clusters and number of data points contained within each cluster.

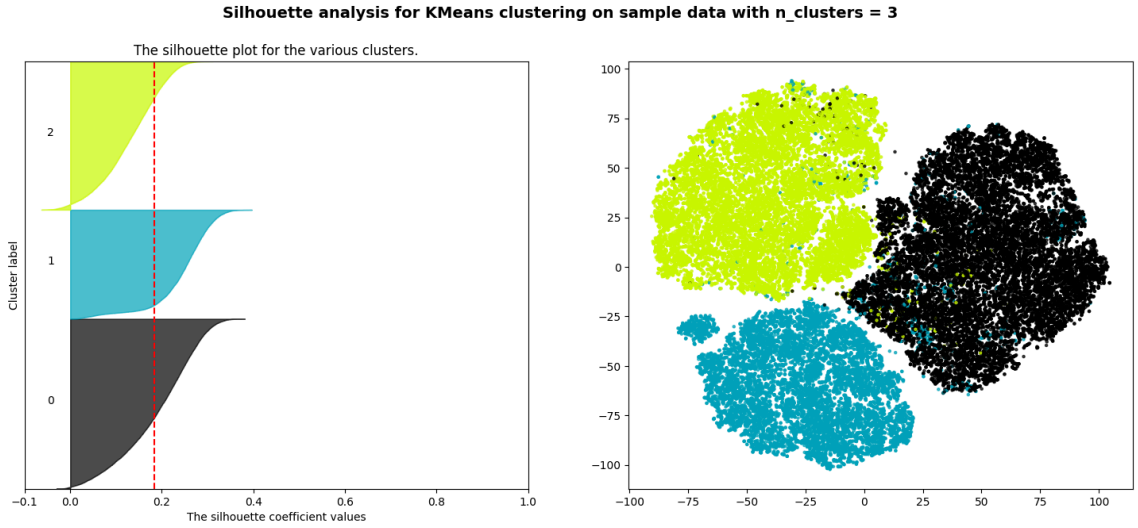
The strongest clustering approach, 10 principal components in 3 clusters produced by K-Means, was manually selected after reviewing the results of the above discussed process. Those clusters were further evaluated by training a scikit-learn random forest regression model on the cluster labels using the reduced survey data set without duplicate binary features, containing 517 features. The most important features were identified and analyzed to understand what each cluster truly represents.

Evaluation:
As noted above, the most significant challenge in identifying useful clusters in the dataset was the large feature space and sparse nature of the data set. Therefore the feature space was reduced utilizing a variety of methods and clustering was performed upon the various reduced datasets.

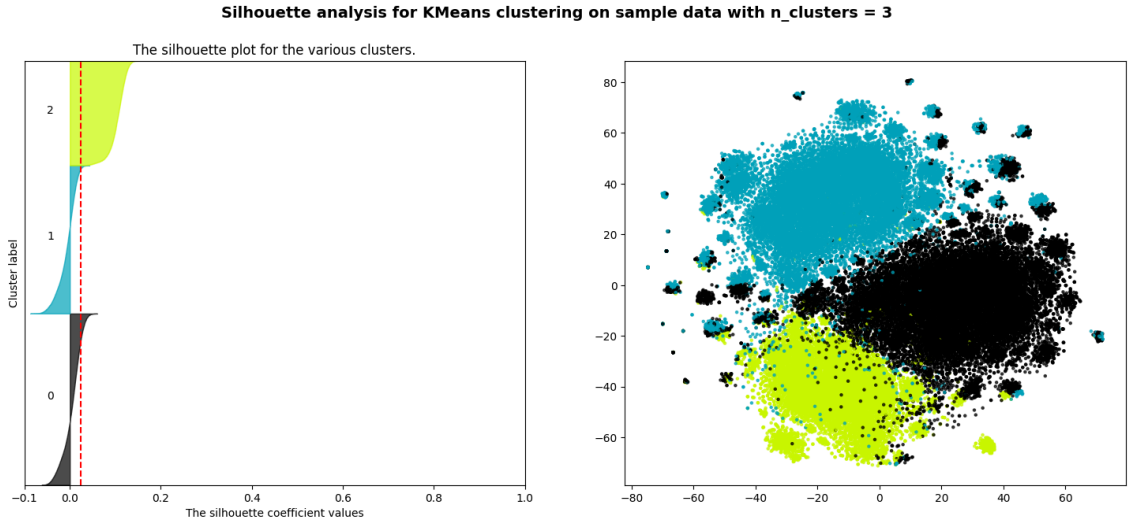
Despite the categorical nature of survey data, applying K-Means clustering can be justified when the data is preprocessed to better fit the algorithm's assumptions. One-hot encoded features, while inherently sparse and categorical, had redundancies removed by dropping one binary variable. Originally sparse data was projected into a continuous feature space using PCA while retaining the variance in the data. This transformation reshapes the dataset into a form where Euclidean distance, utilized by K-Means, becomes an acceptable measure of similarity. While K-Means is not optimal for raw categorical data, it is computationally efficient, widely understood, and works effectively on continuous representations like those produced by PCA.

To evaluate the clustering output, the use of silhouette scores provides a quantitative measure of how well each data point fits within its assigned cluster relative to other clusters, offering a useful statistic for comparing different values of K. Additionally, visualizing the clusters using t-SNE allows for an human interpretable, two-dimensional mapping of the high-dimensional feature spaces, making it easier to assess the cohesion and separation of clusters. While t-SNE does not preserve global geometry, it is effective at revealing local structure, which is helpful when interpreting groupings in complex survey data. Together, this pipeline balances interpretability, efficiency, and practical utility, even if more specialized techniques may yield more accurate results.

The single best clustering approach using K-Means was found to be using 10 Principal Components to identify 3 clusters. The silhouette and t-SNE plots for this combination is below.



Comparatively, the plots for three clusters on the dataset for which PCA retained 80% of the variance were as follows.



The first set of plots are indicative of substantially better cluster identification than the second set of plots.

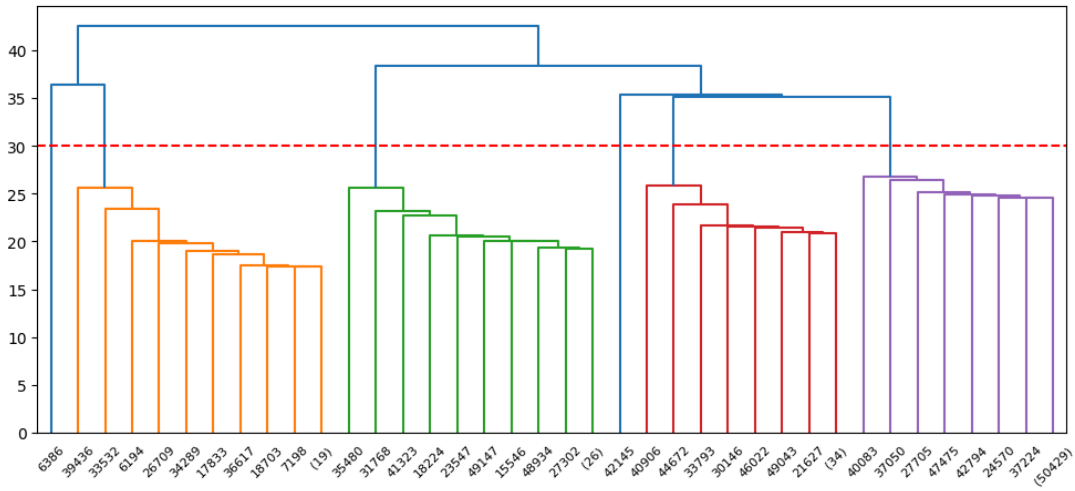
A silhouette score captures the relationship between the intracluster distance of a sample, and the distance to the nearest neighbor cluster. Silhouette scores range from 1 to -1 with a score of 1 being the best and -1 the worst. Therefore, in evaluating the silhouette plots, higher peaks and higher averages are reflective of better clusters. Negative values and lower scores are reflective of weaker clusters. Consistent shapes of the plots for each cluster are also indicative of distinct clusters within the feature space. As the first plot has consistent shapes across the clusters, few negative scores, and significantly taller peaks, it is clearly indicative that the clustering using 10 principal components and 3 clusters via K-Means is superior to the clusters generated by principal components explaining 80% of the variance and 3 clusters via K-means.

The second plot is all samples reduced to 2 dimensions via t-SNE, then plotted and color encoded using the labels assigned by the clustering algorithm. t-SNE captures the local relationships between samples in the dataset. Factors

to consider are sharpness of boundaries between clusters and cohesion within clusters. The t-SNE plot of clusters formed using 10 principal components have three distinct bodies, with clear separation between the bodies, and few outliers within the cluster bodies. Conversely, the second plot has fuzzy boundaries, and the clusters are dispersed across the plot. Comparison of these plots also shows substantially better clustering performance for 10 principal components clustered into 3 clusters by K-Means.

The final method of evaluation used on the K-Means clusters was review of the average silhouette scores across all samples. The average silhouette score for the 3 clusters using 10 principal components was .184. The highest observed average silhouette score. Comparatively, the 3 clusters produced using principal components explaining 80% of the variance achieved an average silhouette score of only .025.

Agglomerative clustering is typically a better approach for clustering highly categorical data than K-Means. Agglomerative clustering is best explored through dendrograms that visualize the "tree" structure of the clustering method. A dendrogram represents cluster distance along the y-axis. Sections of the tree with long vertical jumps are therefore indicative of clusters with more distinct boundaries. Separating the data into clusters using a distance selected within the long vertical jump will therefore typically yield better clusters. Below is a dendrogram produced for the dataset reduced using variance thresholds. The cutline, in red, was the distance selected to identify clusters:

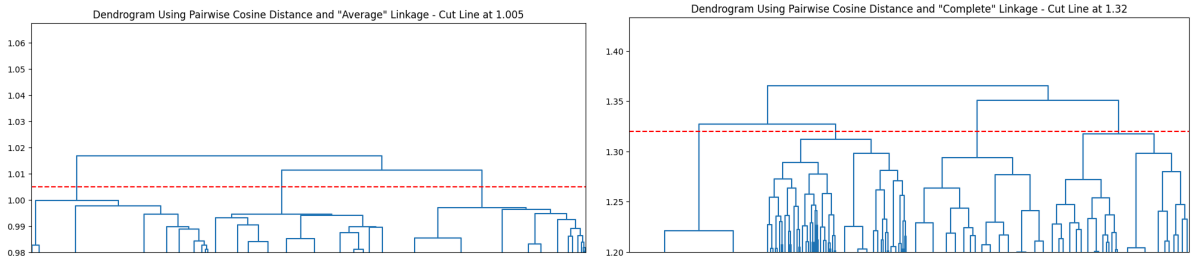


This step was performed manually for each Scipy linkage object fit to each of the datasets discussed previously. Additionally for agglomerative clustering, clustering was attempted upon the entire un-normalized dataset. This was a reasonable attempt as agglomerative clustering is more resistant to ab-normal and feature rich data. This approach did not yield the best results achieved through agglomerative clustering.

Clusters produced by agglomerative clustering were initially reviewed by viewing the number of clusters, and the number of samples within each cluster. Unsurprisingly, the raw data, and the datasets produced for K-Means clustering all yielded poor results that were quickly dismissed in this step of evaluation. The datasets all grouped samples in the center of the feature space early in the tree, generating one large branch and many small branches with very few samples. The dataset with the distance cut shown above, yielded six clusters containing sample counts of [50436, 41, 35, 28, 1, 1].

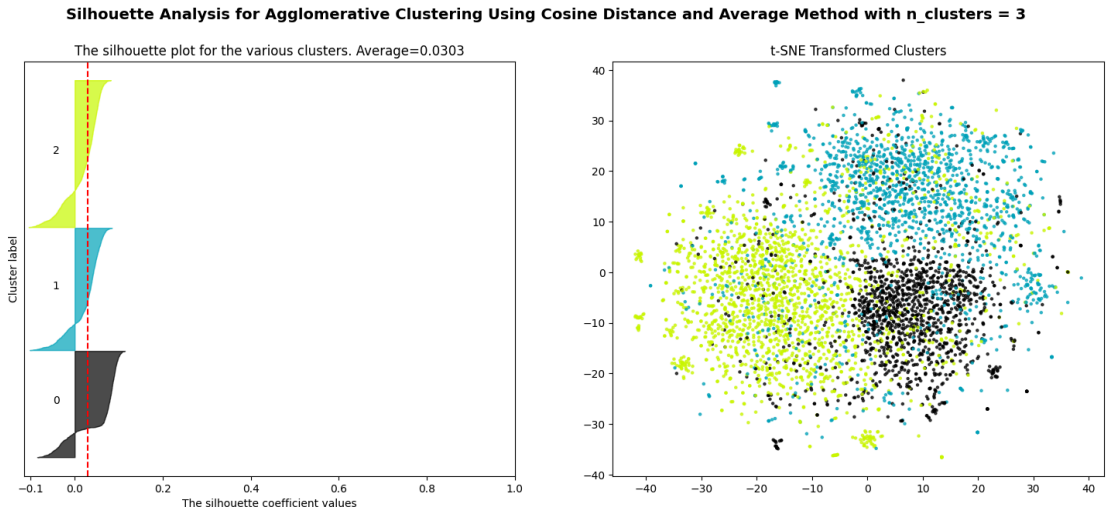
Clustering was also performed using agglomerative clustering on the normalized survey data with binary duplicates dropped containing 517 features with the additional step of first computing a distance matrix using cosine similarity. One tree was then created using the 'average' method and the other using the 'complete' method.

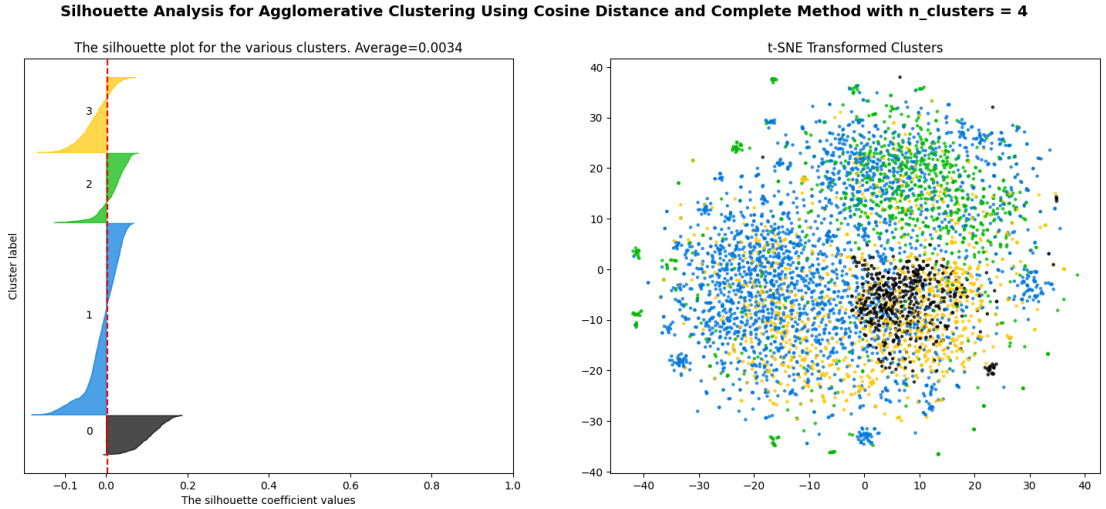
This resulted in the following dendrograms:



This approach yielded clusters containing sample counts of [19574, 17025, 13943] and [25501, 9992, 9860, 5189], far better results than achieved with the other agglomerative approaches.

These clusters were evaluated using silhouette scores and t-SNE as well. However due to computational limitations, silhouette scores were calculated using sub-samples of the dataset. Subsamples of size=5000 were used to produce the below plots:





From this analysis, we can build a comparison table of Unsupervised Learning techniques that is listed in the Appendix. We conclude that K-Means clustering on 10 Principal Components with 3 clusters is the best of the attempted approaches. Sensitivity analysis was performed using different n_cluster values ranging between 2 and 10, and performance evaluated for those other clusters. Average silhouette scores began at .1573 with n_clusters=2, peaked at n_clusters=3, and steadily declined to .1306 at n_clusters = 10. However, visual inspection of the clusters showed that while n_clusters = 2, 4 likely yielded acceptable results, beyond that clusters degraded rapidly.

Additional analysis was performed of the clusters using random forest regression using the 517 feature dataset with dropped binary values and the cluster labels as targets in order to identify the most important features, which represent original survey questions. The ratios of respondents in each cluster who provided a particular answer were inspected. We were able to determine that the model produced three clusters clearly representing: (1) borrowers purchasing a home, (2) borrowers refinancing a home, and (3) borrowers who were either purchasing or refinancing due to some form of personal or financial hardship.

Discussion

In regards to Supervised Learning, we were surprised at the influence of loan term and loan structure on predicting mortgage Beta. Initially, we expected that other features would drive the model. The biggest challenge was the computational costs for modeling. It took a lot of time to undertake the hyperparameter tuning. We responded by sampling the survey data initially. If we had more time, we would try to find more mortgage survey data. This would help us to fine tune the GradientBoosting model, which is “hungry” for data. It looks like there is still some room to run before it plateaus. Additionally, we could remove the loan term/structure features in order to understand other influences on Beta.

In regards to Unsupervised Learning, we were surprised that K-Means clustering on 10 Principal Components with 3 clusters is the best of the attempted approaches. The most significant challenge was the large feature space and the sparse nature of the data set. Therefore, we reduced the feature space using a variety of methods, and then performed clustering on the reduced dataset. If we had more time, we would like to score observations in each cluster (in the holdout partition) using our GradientBoosting model to see if we could identify any interesting patterns.

Ethical Considerations

The mortgage survey data includes questions such as a race, gender, income, and geography. These variables identify protected classes. Ultimately, these features were <u>not</u> useful in developing models nor for creating clusters. However, it would be necessary to remove all variables (and proxy variables) representing protected classes from the data if someone developed and deployed a model in production. It is important to consider the “provenance” of the survey data. People who voluntarily gave answers to the mortgage survey questions would <u>not</u> want their information to be used against them in the future in an unintended manner. Moreover, the heavy influence of the loan term and structure have such an outsized impact on the rate spread that it clouds out any impact that other demographics may play. This does not mean that these discriminations do not exist. However, it may be best to remove these features from the dataset and do a deeper analysis on the remaining ones.

Statement of Work

Paul Stotts focused on Supervised Learning to develop and validate the models.
Michael Schoose focused on Unsupervised Learning to develop and validate the clusters.
John Papazian focused on the original data preparation, YAML files, initial EDA, and other sections.

Appendix: Comparison of Unsupervised Learning Techniques

	K-Means 10 Principal Components N_clusters = 3	**Agglomerative Cosine Distance -Average N_clusters = 3**	**Agglomerative Cosine Distance -Complete N_clusters = 4**
Silhouette Average	.1837	.0303	.0034
Silhouette Scores Mostly Positive	True	False	False
Silhouette Scores Consistency Across Labels	True	True	False
Clear t-SNE Cluster Boundaries	True	False	False
Cohesive t-SNE Clusters	True	False	False
Few t-SNE Outliers	True	False	False

References

[1] <https://www.fhfa.gov/data/national-survey-mortgage-originations-nsmo-public-use-file/dataset>

[2] <https://fred.stlouisfed.org/series/DGS30>

[3] <https://www.freddiemac.com/pmms>

[4] Apaar Sadhwani, Kay Giesecke , Justin Sirignano. "Deep Learning for Mortgage Risk". Journal of Financial Econometrics, Volume 19, Issue 2, Spring 2021, Pages 313–368, <https://doi.org/10.1093/jjfinec/nbaa025>

[5] Sivakumar Pillai, Jennifer Woodbury, Nikhil Dikshit, Avery Leider, and Charles Tappert. "Machine Learning Analysis of Mortgage Credit Risk". Proceedings of the Future Technologies Conference, October 2019, Pages 107–123, <https://doi.org/10.1007/978-3-030-32520-6_10>

[6] Vikram Ojha and JeongHoe Lee. "Default analysis in mortgage risk with conventional and deep machine learning focusing on 2008–2009." Digital Finance 3, 2021, Pages 249–271. <https://doi.org/10.1007/s42521-021-00036-4>

[7] <https://www.fhfa.gov/data/national-survey-mortgage-originations-nsmo-public-use-file/dataset>

[8] <https://www.fhfa.gov/document/NSMO-Technical-Report-v50.pdf>

[9] <https://www.fhfa.gov/sites/default/files/2024-06/v50-Appendix-C.pdf>